

Project Report

PROJECT TITLE

TaskFlow

A Cross-Platform To-Do Mobile Application built with React Native & Expo CLI

Submitted By	Devraj Panchal
Enrollment Number	2305101270066
Subject	Cross Platform Development (CPD)
Technology	React Native (Expo CLI)
Platforms Supported	Android, iOS & Web
Guided By / Faculty	
Academic Year	
Submission Date	September 2026

Table of Contents

1. Introduction / Project Overview	3
2. Objectives	3
3. Features	4
4. Technology Stack	5
5. System Requirements	5
6. Project Structure	6
7. Application Screens	7
8. Working of the Application	10
9. Key Code Snippets	10
10. Component Description	12
11. Testing & Test Cases	12
12. Limitations & Future Enhancements	13
13. GitHub Repository Details	14
14. Conclusion	15
15. References	15

1. Introduction / Project Overview

TaskFlow is a small but fully functional *to-do list* mobile application developed as part of the Cross Platform Development (CPD) subject. The goal of the project is to demonstrate how a single JavaScript/React codebase can be used to build a native-quality mobile application that runs unchanged on **Android, iOS and the Web** using the **React Native** framework and the **Expo CLI** toolchain.

The application allows users to quickly record their daily tasks, mark them as completed, filter between pending and finished work, and delete tasks that are no longer needed. All data is stored **locally on the device** using AsyncStorage, so the app works fully offline and the user's tasks remain saved even after the app is closed and reopened.

The project intentionally keeps the scope small so that every concept — components, state management with hooks, list rendering, user input, event handling and on-device storage — can be understood clearly, while still delivering a genuinely useful and polished app.

What is React Native & Expo?

React Native is an open-source framework created by Meta that lets developers build mobile apps using React and JavaScript, rendering real native UI components. **Expo** is a platform and CLI built around React Native that removes the need for complex native build setup: a developer can start an app with one command and run it on a physical phone instantly by scanning a QR code with the **Expo Go** app.

2. Objectives

- Build a complete, runnable cross-platform mobile application using React Native and Expo CLI.
- Demonstrate the use of core React Native components: `View` , `Text` , `TextInput` , `FlatList` , `Pressable` and `TouchableOpacity` .
- Manage application state using React hooks (`useState` and `useEffect`).
- Implement offline data persistence with AsyncStorage.
- Create a clean, responsive and touch-friendly user interface from a shared codebase for Android, iOS and Web.
- Organise the project with reusable components and host the complete source code on GitHub for evaluation.

3. Features

Feature	Description
Add Task	Type a task in the input field and tap the blue + button (or press Enter) to add it to the top of the list.
Complete Task	Tapping the circular checkbox marks a task as done — it turns green with a tick and the task text gets a strikethrough.
Delete Task	The red ✕ button removes an individual task immediately.
Filter Tabs	All / Active / Done tabs filter the list and display a live count for each category.
Clear Completed	A shortcut at the bottom removes all finished tasks in one tap.
Offline Persistence	Tasks are saved to the device with AsyncStorage and restored automatically on the next launch — no internet or login required.
Empty State	A friendly screen with an icon guides the user when there are no tasks to show.
Accessibility	All buttons and checkboxes expose accessibility roles and labels for screen readers.

4. Technology Stack

Tool / Library	Version	Purpose
Expo SDK	~57.0.24	Cross-platform toolkit, dev server and CLI
React Native	0.86.3	Framework providing native UI components
React	19.2.3	Component model and hooks (state & effects)
AsyncStorage	2.2.0	Asynchronous, persistent key-value storage on the device
react-native-safe-area-context	4.x	Handles notches and safe areas (top/bottom)
react-native-web / react-dom	latest	Allow the same app to run in a web browser
Node.js / npm	Node 18+	Runtime and package manager used for development
Expo Go	latest	Mobile app used to run the project on a physical device
Editor	Any (VS Code recommended)	Writing and editing the source code

5. System Requirements

Software (development machine)

- Operating System: Windows 10/11, macOS, or Linux (Ubuntu)
- Node.js (LTS) and npm installed
- A terminal/command prompt and a code editor (VS Code recommended)
- The Expo Go app on an Android or iOS phone (optional: Android Studio emulator)

Hardware

- 4 GB RAM minimum (8 GB recommended), a normal dual-core processor and ~500 MB free disk space
- An Android/iOS smartphone with a camera (to scan the QR code)

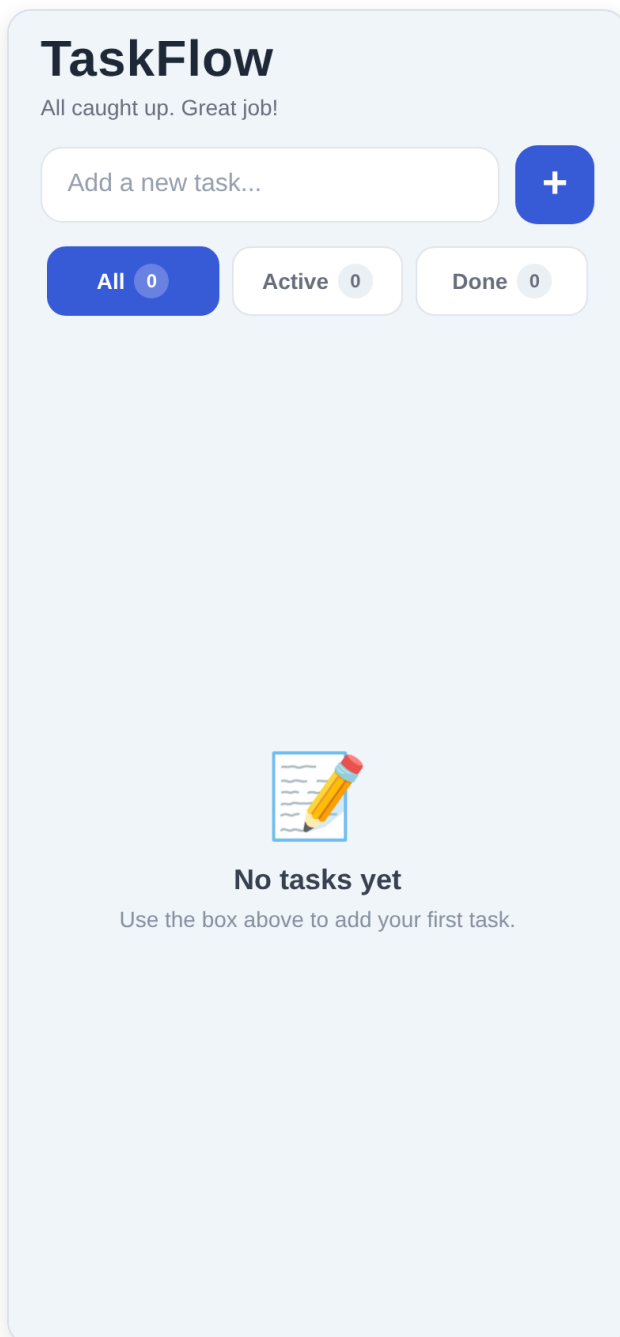
6. Project Structure

```
taskflow/
├─ App.js           # Main screen: state, handlers and full layout
├─ index.js         # Entry point that registers App
├─ app.json         # Expo configuration (name, icon, platforms)
├─ package.json     # Dependencies and npm scripts
├─ package-lock.json # Locked dependency versions
├─ README.md        # Project documentation
├─ assets/          # App icon, splash and favicon images
│   └─ icon.png
│   └─ splash-icon.png
│   └─ favicon.png
│   └─ android-icon-*.png
└─ components/
    └─ TaskItem.js   # Reusable single task row component
    └─ FilterTabs.js # Reusable All / Active / Done tabs
```

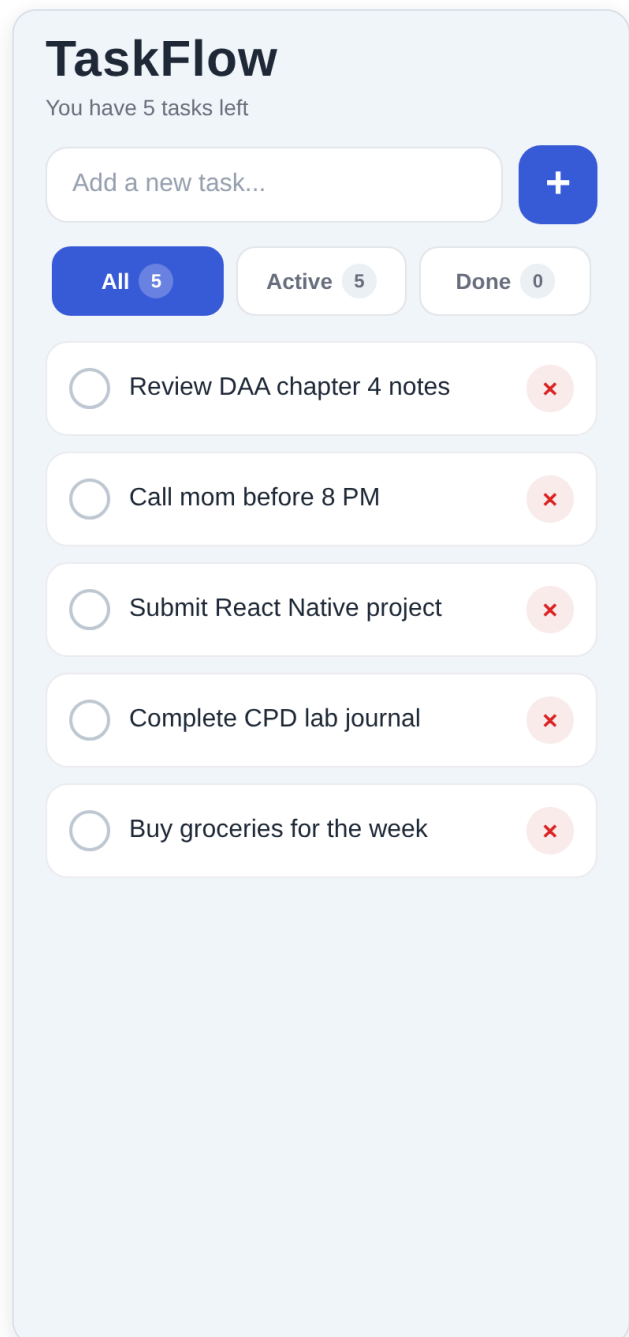
The `node_modules/` folder (downloaded dependencies) is excluded from GitHub through `.gitignore` ; evaluators recreate it with `npm install` .

7. Application Screens

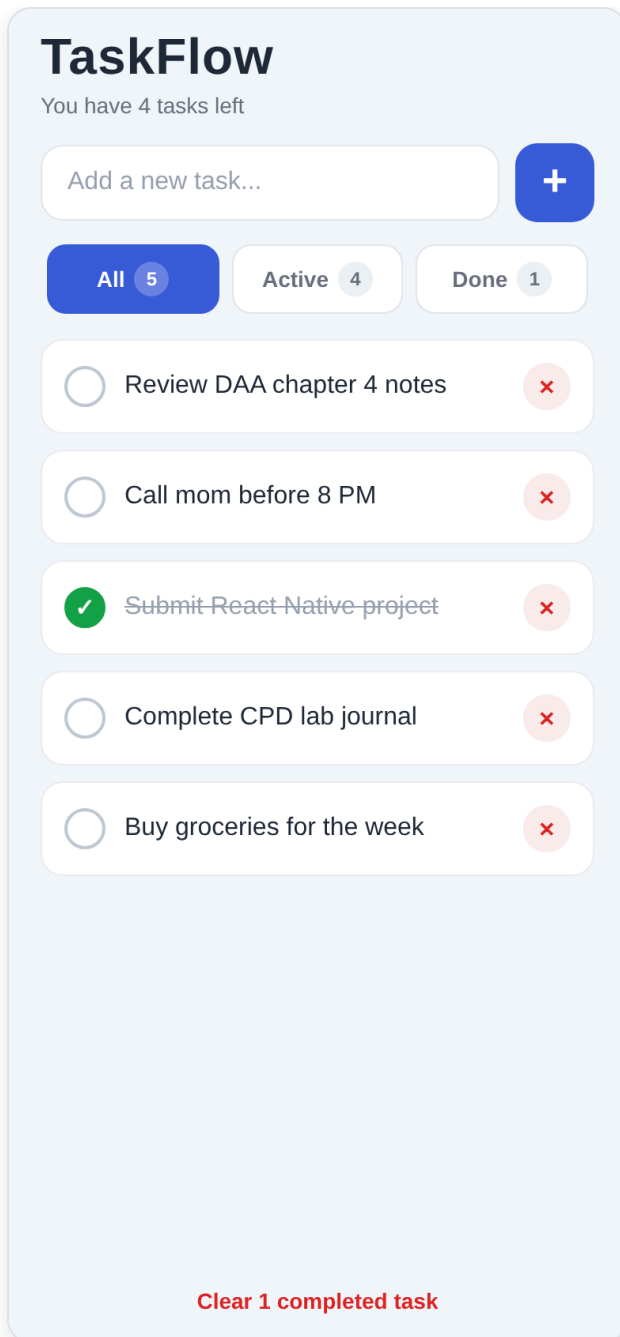
The screens below were captured from the running application at a 390×844 phone viewport.



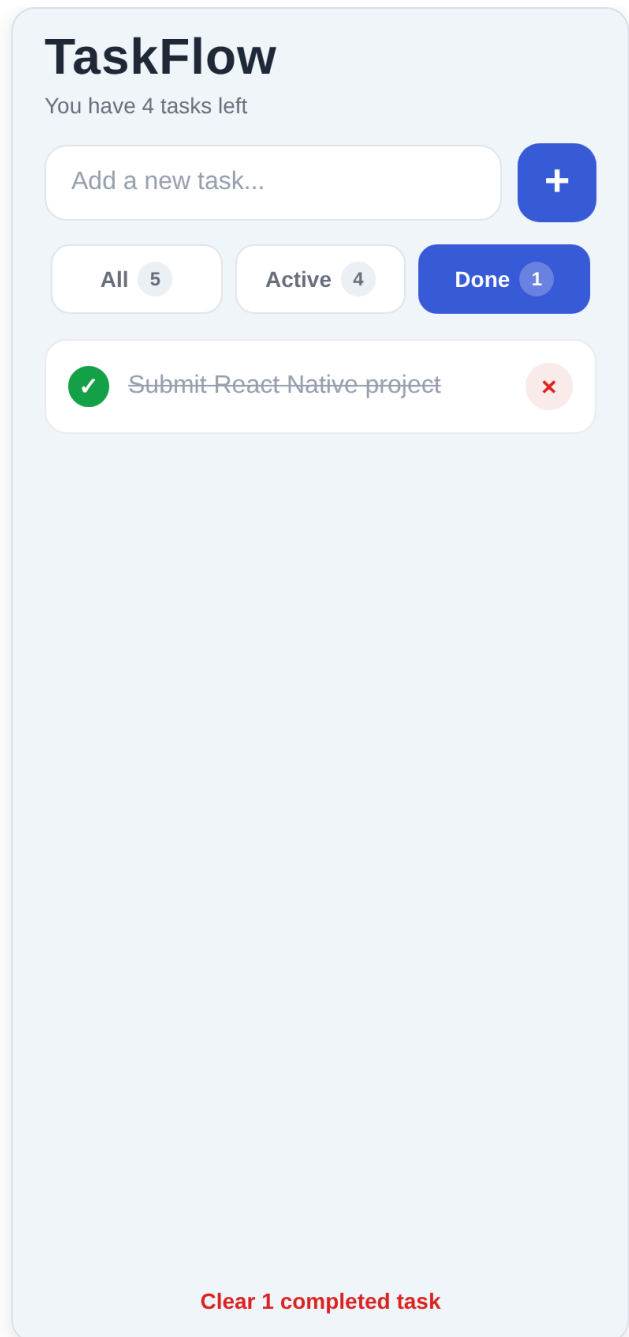
Screen 1 — Empty State: when no tasks exist, the app shows a friendly prompt to add the first task.



Screen 2 — All Tasks: newly added tasks appear as cards with a checkbox and a delete button; tabs show live counts.



Screen 3 — Task Completed: tapping the circle marks it done (green tick + strikethrough) and the "Clear completed" shortcut appears.



Screen 4 — Done Filter: the Done tab lists only completed tasks.

TaskFlow

You have 4 tasks left



All 5

Active 4

Done 1



Review DAA chapter 4 notes



Call mom before 8 PM



Complete CPD lab journal



Buy groceries for the week



Clear 1 completed task

Screen 5 — Active Filter: the Active tab hides finished tasks and shows only pending work.

8. Working of the Application

The application follows a simple, one-way data flow:

1. **Launch:** `App` renders and the first `useEffect` reads `@taskflow/tasks_v1` from `AsyncStorage`; saved tasks (if any) are loaded into state.
2. **Add:** the user types text and taps `+`. A new task object `{ id, text, completed, createdAt }` is prepended to the `tasks` array and the keyboard is dismissed.
3. **Toggle:** tapping a checkbox maps over the tasks and flips the matching task's `completed` value.
4. **Delete / Clear:** tasks are removed from the array by id (or all completed tasks at once).
5. **Filter:** the active tab determines which tasks the `FlatList` receives from a filtered copy of the array.
6. **Auto-save:** a second `useEffect` watches `tasks` and writes the updated list back to `AsyncStorage` after every change.

9. Key Code Snippets

9.1 Adding a new task (`App.js`)

```
const addTask = () => {
  const trimmed = text.trim();
  if (trimmed.length === 0) return; // ignore empty input

  const newTask = {
    id: `${Date.now()}-${Math.random().toString(36).slice(2, 8)}`,
    text: trimmed,
    completed: false,
    createdAt: Date.now(),
  };

  setTasks((prev) => [newTask, ...prev]);
  setText('');
  Keyboard.dismiss();
};
```

9.2 Completing and deleting tasks

```
const toggleTask = (id) => {
  setTasks((prev) =>
    prev.map((task) =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  );
};

const deleteTask = (id) => {
  setTasks((prev) => prev.filter((task) => task.id !== id));
};
```

9.3 Loading tasks on launch and saving on every change

```
// Load once
useEffect(() => {
```

```

(async () => {
  try {
    const raw = await AsyncStorage.getItem(STORAGE_KEY);
    if (raw !== null) setTasks(JSON.parse(raw));
  } catch (error) {
    console.warn('Could not load tasks:', error);
  } finally {
    setLoaded(true);
  }
})();
}, []);

// Persist after every change (once loading has finished)
useEffect(() => {
  if (!loaded) return;
  AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(tasks))
    .catch((error) => console.warn('Could not save tasks:', error));
}, [tasks, loaded]);

```

9.4 Rendering the list with FlatList

```

<FlatList
  data={visibleTasks}
  keyExtractor={({item}) => item.id}
  renderItem={({item}) => (
    <TaskItem task={item} onToggle={toggleTask} onDelete={deleteTask} />
  )}
  ListEmptyComponent={<EmptyState />}
/>

```

10. Component Description

Component	Responsibility
App (App.js)	Root screen. Holds all state (<code>tasks</code> , <code>text</code> , <code>filter</code> , <code>loaded</code>), defines the add/toggle/delete/clear handlers, performs persistence and composes the header, input bar, filter tabs and list.
TaskItem (components/TaskItem.js)	A presentational row for one task. Renders the circular checkbox, the task text (with strikethrough when done) and the red delete button. Receives <code>task</code> , <code>onToggle</code> and <code>onDelete</code> as props.
FilterTabs (components/FilterTabs.js)	Renders the All / Active / Done tabs. Receives the tab definitions, the active tab, the counts object and an <code>onChange</code> callback; highlights the selected tab.

11. Testing & Test Cases

The application was tested on the web build and through the Expo bundler for Android (the production bundle compiled with 0 errors). The following functional test cases were executed:

#	Test Scenario	Expected Result	Result
1	First launch	App opens, shows header, input, tabs (All 0) and the empty-state message.	Pass
2	Add a valid task	Task appears at the top of the list, input clears, All/Active counts increase.	Pass
3	Add empty / spaces only	Nothing is added to the list.	Pass
4	Tap task checkbox	Circle turns green with a tick, text is struck through, Done count increases, Active decreases.	Pass
5	Tap checkbox again	Task returns to the active state.	Pass
6	Tap delete (X)	That task is removed and all counts update.	Pass
7	Switch filter tabs	All shows every task, Active only pending tasks, Done only completed tasks.	Pass
8	Restart the app	Previously entered tasks are restored from AsyncStorage automatically.	Pass
9	Clear completed	Only completed tasks are removed; active tasks remain untouched.	Pass
10	Android bundle build	<code>npx expo export --platform android</code> completes with no errors (597 modules bundled).	Pass

12. Limitations & Future Enhancements

Current Limitations

- Tasks are stored only on the single device; there is no cloud backup or sync between devices.
- Tasks cannot be edited after being added (they must be deleted and re-created).
- No due dates, reminders, categories or priority levels.
- Only a light colour theme is provided.
- No user authentication or multi-user support.

Future Enhancements

- Tap-to-edit task text.
- Due dates with local notifications/reminders (Expo Notifications).
- Categories, tags and colour coding; drag-to-reorder tasks.
- Dark mode and multiple language support.
- Cloud synchronisation (Firebase/Supabase) with login.

13. GitHub Repository Details

Repository URL: <https://github.com/kai-ryuzen/taskflow>

Visibility: Public • **Default branch:** main • **Owner:** kai-ryuzen

The complete source code has been uploaded to GitHub, including `App.js`, the `components/` folder, Expo configuration, assets, `package.json`, `package-lock.json`, `.gitignore` and this project's README. The `node_modules` folder is intentionally excluded and can be regenerated.

How to clone and run the project

```
# 1. Clone the repository
git clone https://github.com/kai-ryuzen/taskflow.git

# 2. Move into the project folder
cd taskflow

# 3. Install dependencies
npm install

# 4. Start Expo and scan the QR code with Expo Go
npx expo start
```

Repository Contents

`App.js`, `index.js`, `app.json`, `package.json`, `package-lock.json`, `README.md`, `.gitignore`, `LICENSE`, `AGENTS.md`, `assets/` (icons & splash), `components/` (`TaskItem.js`, `FilterTabs.js`)

14. Conclusion

TaskFlow successfully demonstrates the core idea of cross-platform development: writing one small, well-structured React Native codebase with the Expo CLI and running it natively on Android, iOS and the Web. Despite its small size, the project covers the essential building blocks of real mobile applications — reusable components, hook-based state management, user input, scrollable lists, touch handling, accessibility and persistent on-device storage. Building this project provided practical, hands-on understanding of the React Native / Expo workflow, from project creation and development through bundling, documentation and publishing the source code on GitHub. The app can easily be extended with the enhancements listed above as learning progresses.

15. References

1. React Native Official Documentation — <https://reactnative.dev/docs/getting-started>
2. Expo Official Documentation — <https://docs.expo.dev/>
3. Expo Go — <https://expo.dev/go>
4. AsyncStorage Documentation — <https://react-native-async-storage.github.io/async-storage.dev/>
5. React Hooks Introduction — <https://react.dev/learn/state-a-components-memory>
6. GitHub Docs (Repositories) — <https://docs.github.com/en/repositories>